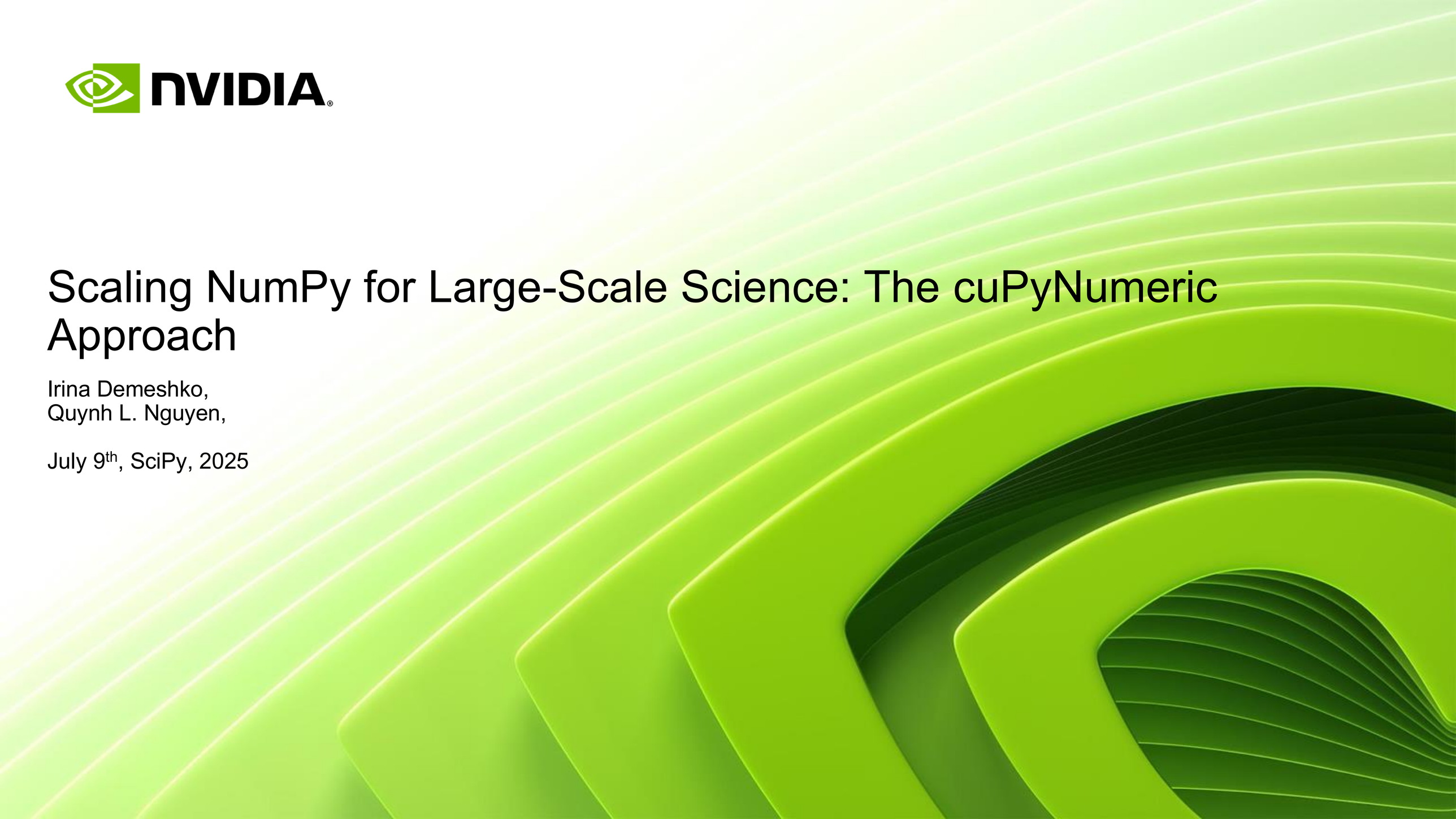


Scaling NumPy for Large-Scale Science: The cuPyNumeric Approach

Irina Demeshko,
Quynh L. Nguyen,

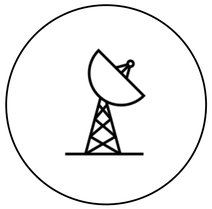
July 9th, SciPy, 2025

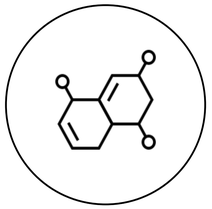
NumPy Powers Scientific Computing

5M
Developers

300M
Downloads/Month

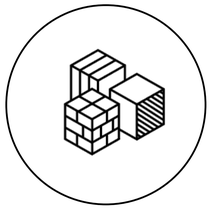
32K
GitHub Applications

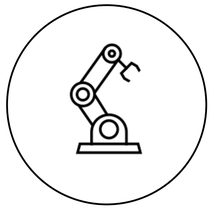

Astronomy

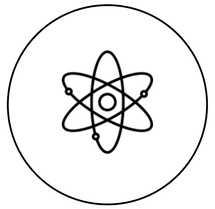

Biology

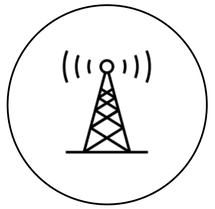

Chemistry


Climate


Materials

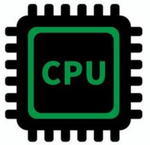

Engineering


Physics


Signal Processing

Larger-scale Problems

- Memory Limitations: too much data to fit to a single node/single GPU



- Time Limitations: computation time is taking very long time

Use cases for scaled NumPy:

Scientific Simulations

Climate modeling, fluid dynamics

GPU-Accelerated AI/ML

Data preprocessing of massive datasets

Financial Monte Carlo

Millions of parallel stochastic simulations

Genomics & Bioinformatics

DNA sequence alignment, matrix operations

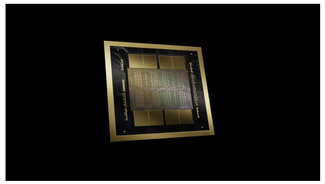
NumPy at Scale

NumPy

```
import numpy as np
m, n, k = 1000, 1000, 1000
A = np.random.rand(m,k).astype(float)
B = np.random.rand(m,k).astype(float)
res=A@B
print(res)
```

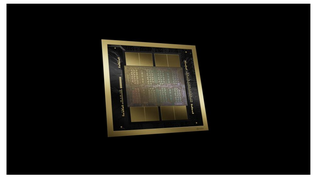
CuPy

```
import cupy as cp
m, n, k = 1000, 1000, 1000
A = cp.random.rand(m,k).astype(float)
B = cp.random.rand(m,k).astype(float)
res=A@B
print(res)
```



cuPyNumeric

```
import cupynumeric as np
m, n, k = 1000, 1000, 1000
A = np.random.rand(m,k).astype(float)
B = np.random.rand(m,k).astype(float)
res=A@B
print(res)
```



Scaling NumPy with cuPyNumeric

```python
import cupynumeric as np
A = np.random.rand(m,k).astype(np.float32)
B = np.random.rand(k,n).astype(np.float32)

print(A@B)
```

Simple python code w/ NumPy
No partitioning code
No MPI code

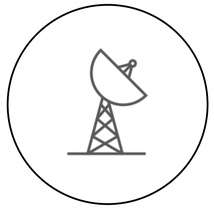
Scales to multi-GPU multi-Node at run time

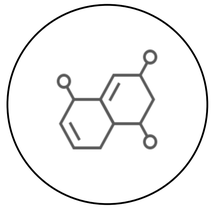
```
(legate) bod@dgx-1:~$ legate --nodes 64 --gpus 8 matmul.py
```

Get the science done faster!

- Write code in **simply Python w/ NumPy**.
- Scaling from single CPU core to a multi-GPU multi-node supercomputer with thousands of GPUs. Zero-code-change approach of cuPyNumeric is suitable for NumPy code that follows best practices and operates on large data arrays. For other code, some additional modifications may be necessary to achieve good performance.
- **Use HPC clusters without HPC programming** or waiting for other engineers to rewrite w/ MPI (other libraries) to scale up to thousand nodes.
- Drop-in replacement library for NumPy and **expanding to SciPy, and other scientific libraries**.

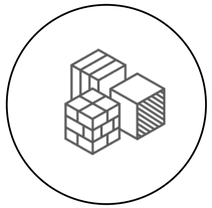
*

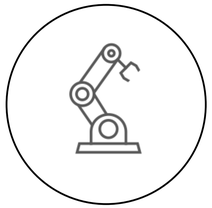

Astronomy

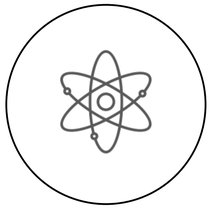

Biology

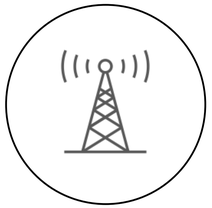

Chemistry

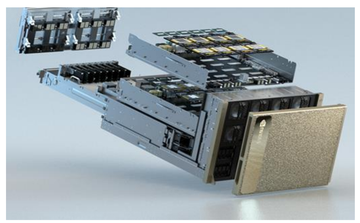

Climate

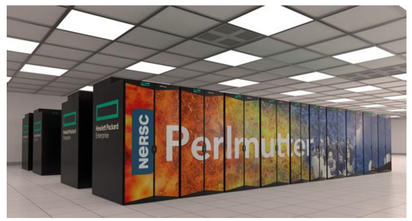

Materials


Engineering


Physics


Signal Processing



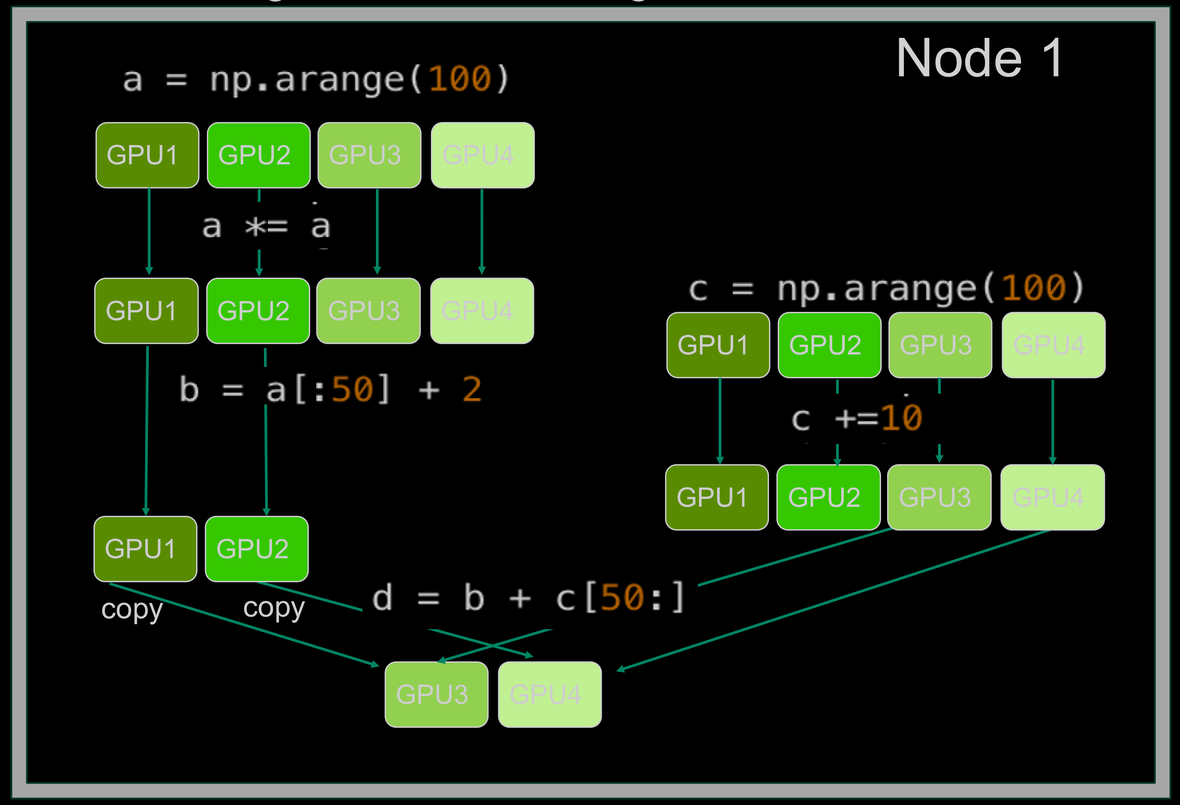


Automatic parallel execution and data management in NVIDIA cuPyNumeric

Program order

```
import  cupynumeric as np

a = np.arange(100)
a *= a
b = a[:50] + 2
c = np.arange(100)
c +=10
d = b + c[50:]
```

Program execution diagram



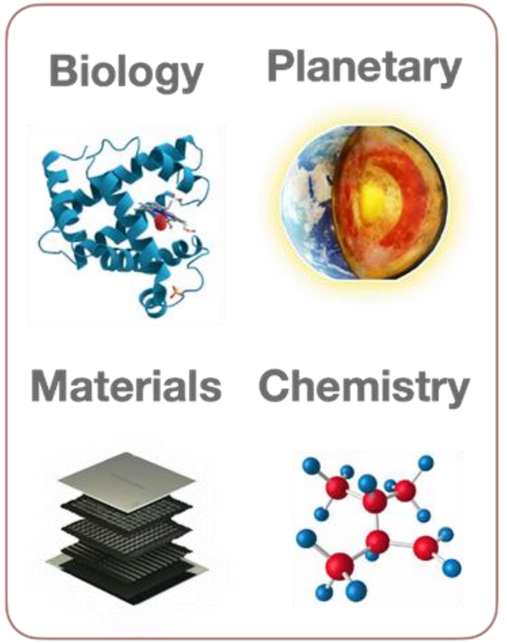
Linac Coherent Light Source at SLAC National Accelerator Laboratory

World's first hard X-ray Free Electron Laser

SLAC X-ray Free Electron Laser

- 3-km long tunnel under I-280 and close to Stanford campus
- Access angstrom-length-scales and electronic movements
- Unravel new scientific insights in matter

Applications



SLAC

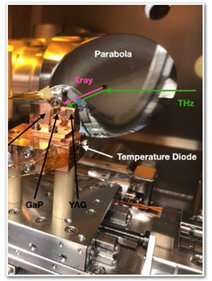
NVIDIA

Time-resolved X-ray experiment schematic timeline

~7 months of data analysis

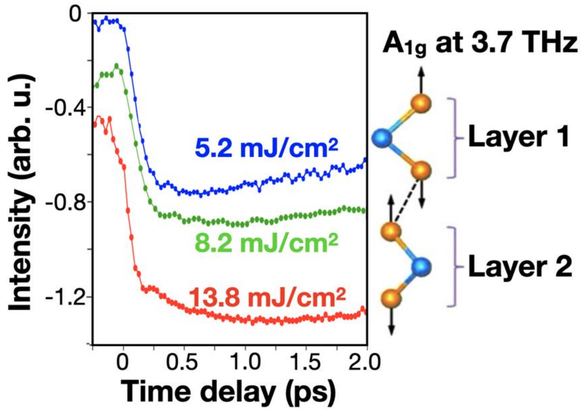
0. Setup Experiment

~1-year



1. Data Collection

2. Data Processing & Analysis



SLAC

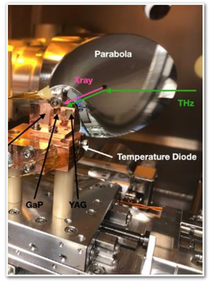
NVIDIA

Time-resolved X-ray experiment schematic timeline

~7 months of data analysis

0. Setup Experiment

~1-year

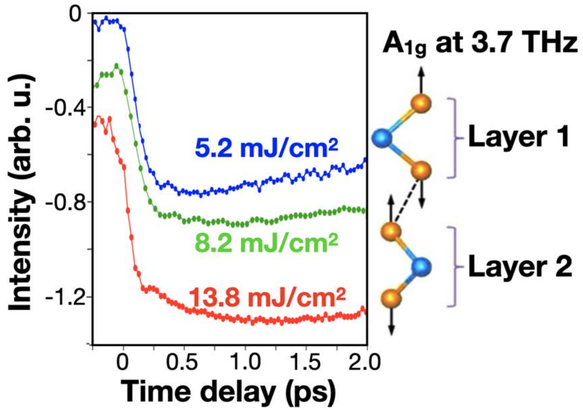


Multiyear to Publications!

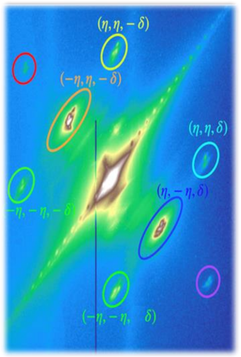
4. Decision Making

1. Data Collection

2. Data Processing & Analysis



3. Important Insights



SLAC

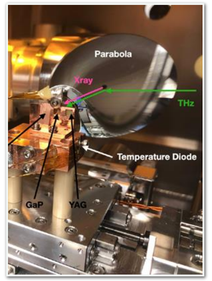
NVIDIA

Time-resolved X-ray experiment schematic timeline

~7 months of data analysis

0. Setup Experiment

~1-year

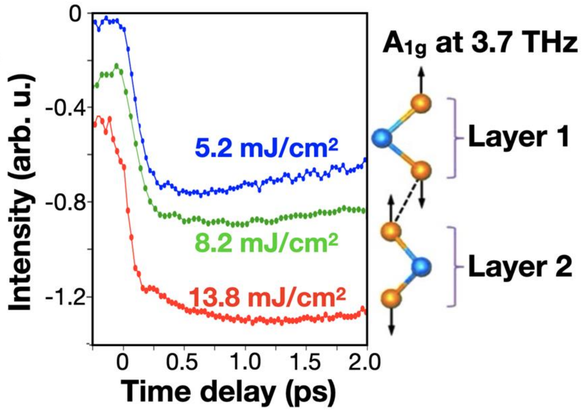


Multiyear to Publications!

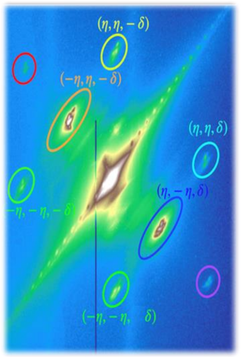
4. Decision Making

1. Data Collection

2. Data Processing & Analysis



3. Important Insights

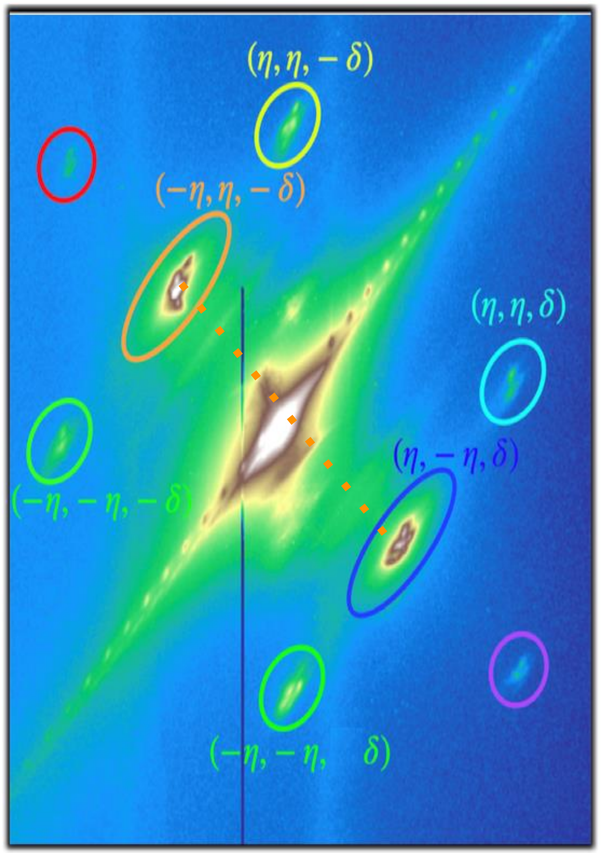


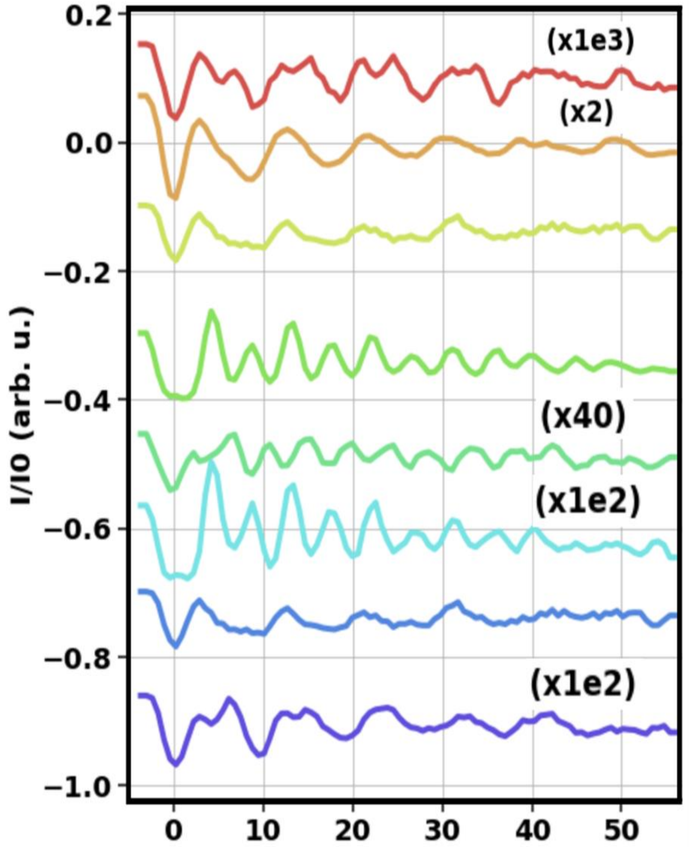
SLAC

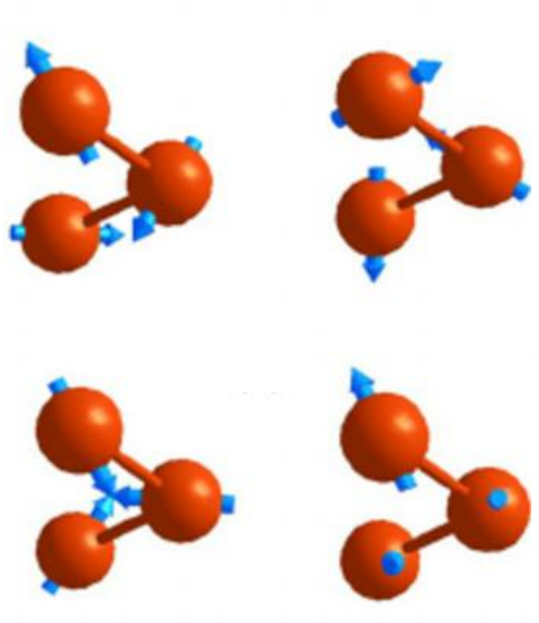
NVIDIA

Multidimensional Data Structure

Measure Unique Material Properties to Reveal Formation of new Phases in Materials

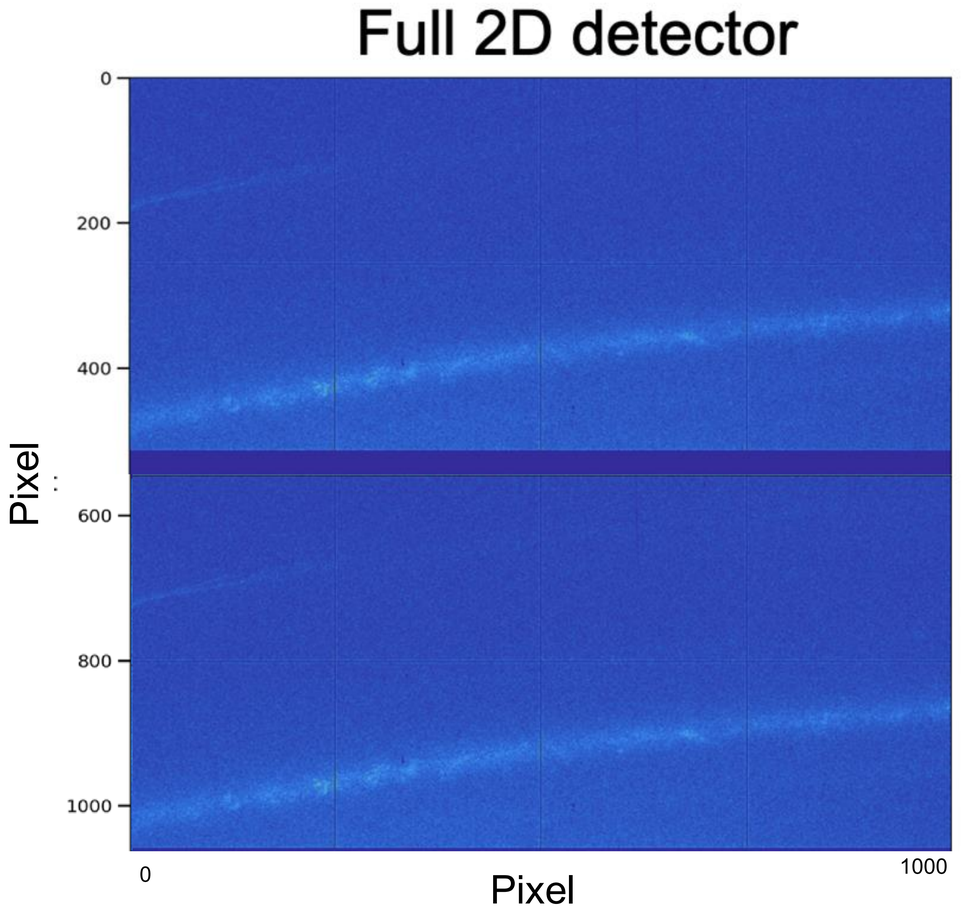






QL Nguyen et al. Physical review letters 131 (7), 076901 (2023)

Where is the signal?

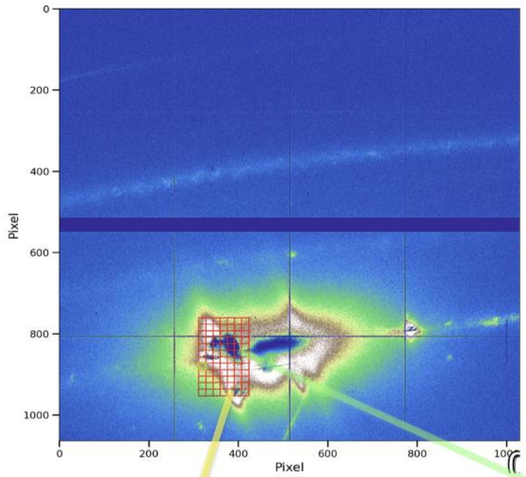


Experiments with 1% signal

Not visible by eye. Calculations predicted the signal here. But where is the signal?

Parallelized Full Detector with cuPyNumeric Pixel-Analysis

from 7 months of data analysis to less than a day

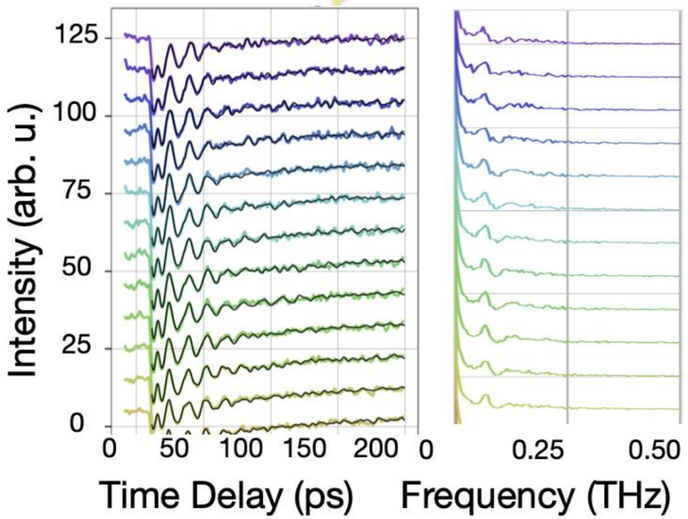


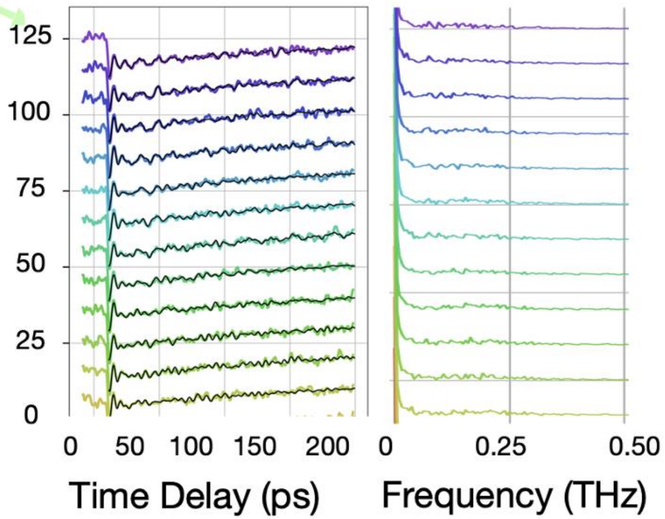
Full detector analysis enables automatic

- Signal detection
- Signal analysis
- Data compression/reduction
- Fast experimental feedback
- Smart-experimental steering
- Faster science discovery

Fit

Integrated signal from each tile





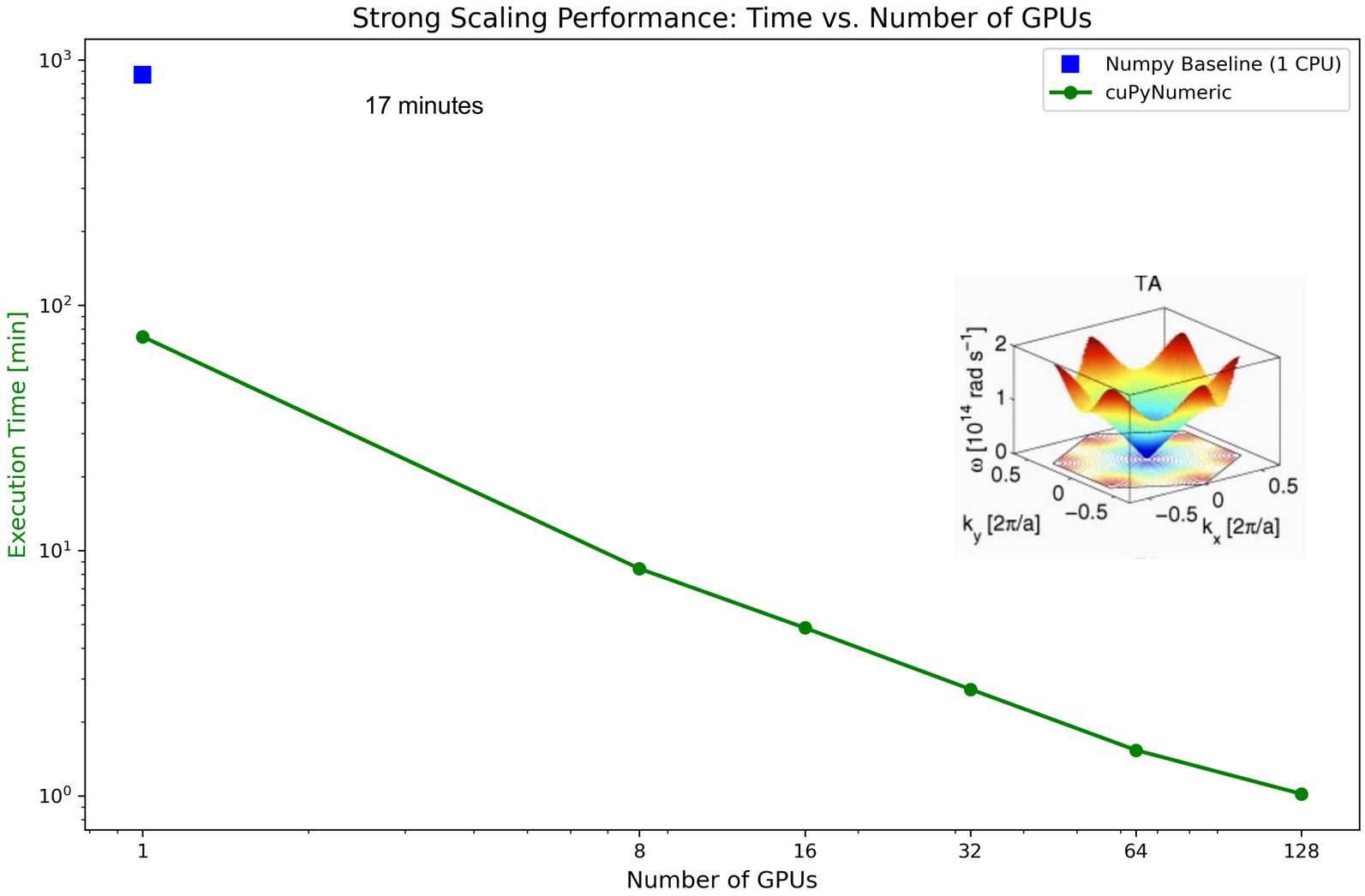
DOE Perlmutter

NVIDIA DGX

SLAC

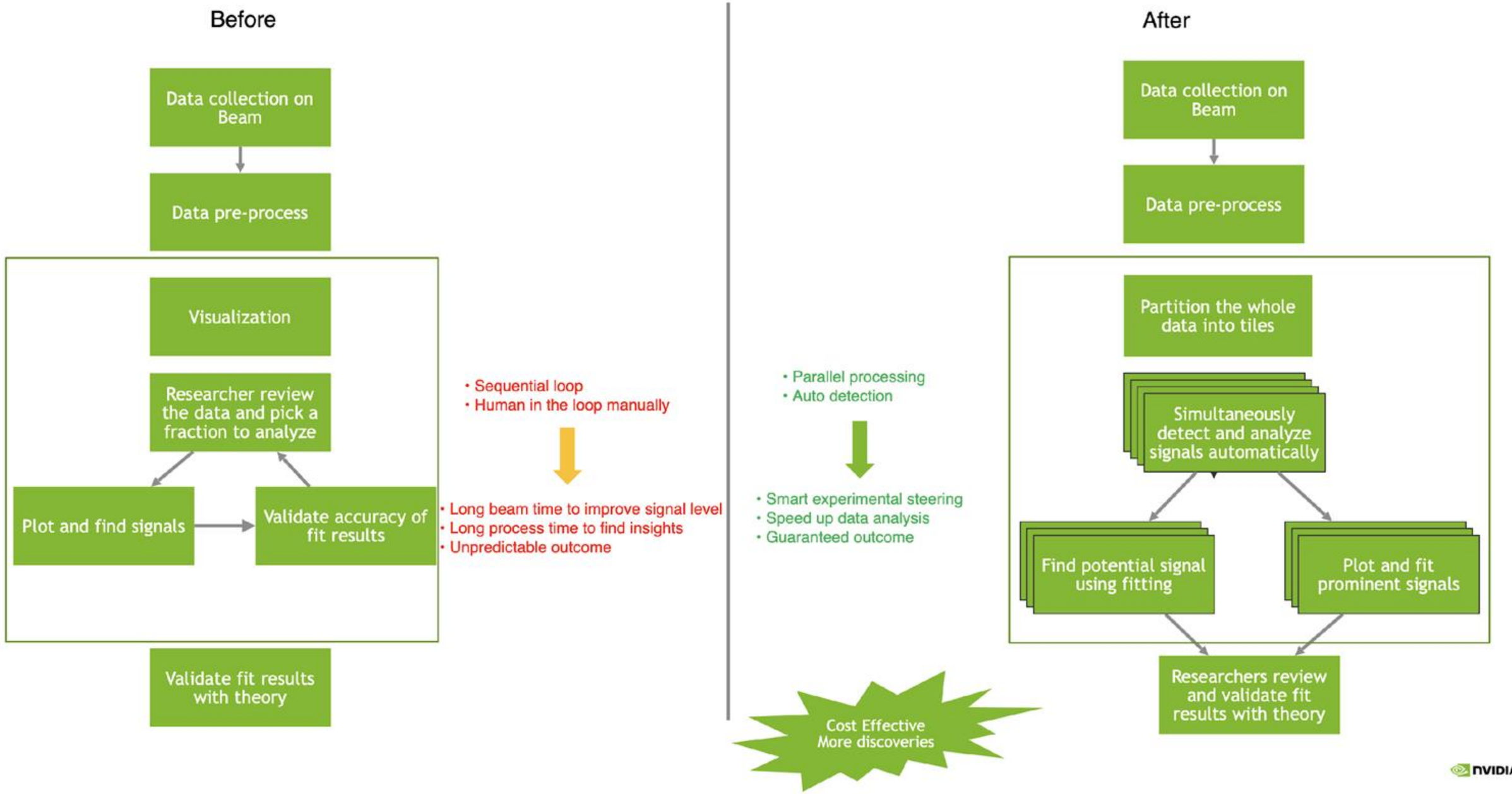
NVIDIA

Performance results



NVIDIA DGX H100

New LCLS Scientific User Experiment workflow



NumPy/cuPyNumeric best practices

cuPyNumeric's zero-code-change approach is suitable for NumPy code that follows best practices and operates on large data arrays. For other code, some additional modifications may be necessary to achieve good performance.

Use cuPyNumeric or NumPy arrays, AVOID native lists

```
x = [1, 2, 3]
y = []
for val in x:
    y.append(val + 2)
```

```
y = np.array(x)
y = x + 2
```

Use array-based operations, AVOID loops with indexing

```
for i in range(ny):
    for j in range(nx):
        if (y[j, i] < tol):
            x[j, i] = const
        else
            x[j, i] = 1.0 - const
```

```
cond = y < tol
x[cond] = const
x[~cond] = 1.0 - const
```

Use boolean masks, AVOID advanced indexing

```
indices = np.nonzero(h < 0)
x[indices] = y[indices]
```

```
cond = h < 0
x[cond] = y[cond]
```

& more

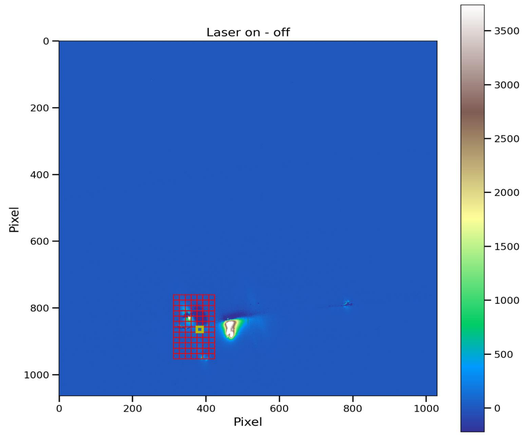
For more details see https://docs.nvidia.com/cupynumeric/latest/user/practices.html

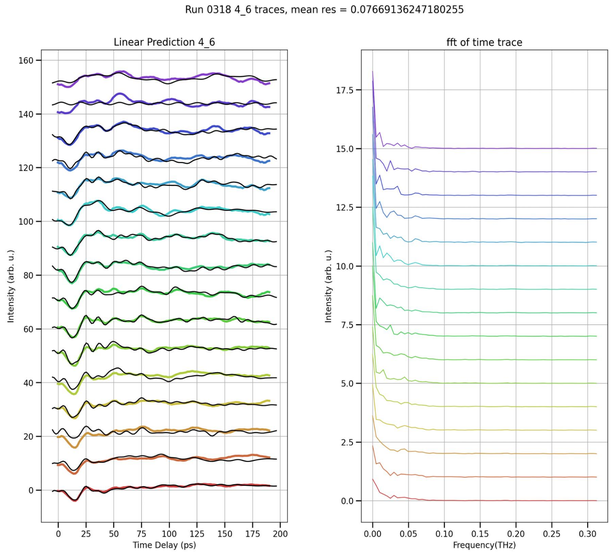
Advanced cuPyNumeric

Python Tasks

- **cuPyNumeric** performs best on code that includes a sufficient number of array-level operations and operates on arrays large enough to be distributed across multiple resources (e.g., GPUs or nodes).
- If your workload does not naturally fall into this category, cuPyNumeric still provides APIs for parallel execution of independent parts of your code. For example, **Python task API**, which enables explicit parallelism for non-array-centric workloads.

```python
import cupynumeric as np
import cupy as cp

# Tiled task: modifies tiles of the array
tile_shape = (10000, 50)
task_shape = (1, 20)

@cupy_task(
    arg_roles={"tile": "in", "new_tile": "out"},
    partitioning_type="tiled",
    task_shape=task_shape,
    tile_shape=tile_shape
)
def modify_tile(tile, new_tile):
    print("Inside modify_tile")
    for _ in range(10):
        new_tile[:] = (cp.power(tile, 2) * 100 - 3) / 2

# Prepare data
array = np.random.rand(10000, 1000)
new_array = np.zeros_like(array)

# Execute tiled modify task
modify_tile(array, new_array)
```





Legate Ecosystem

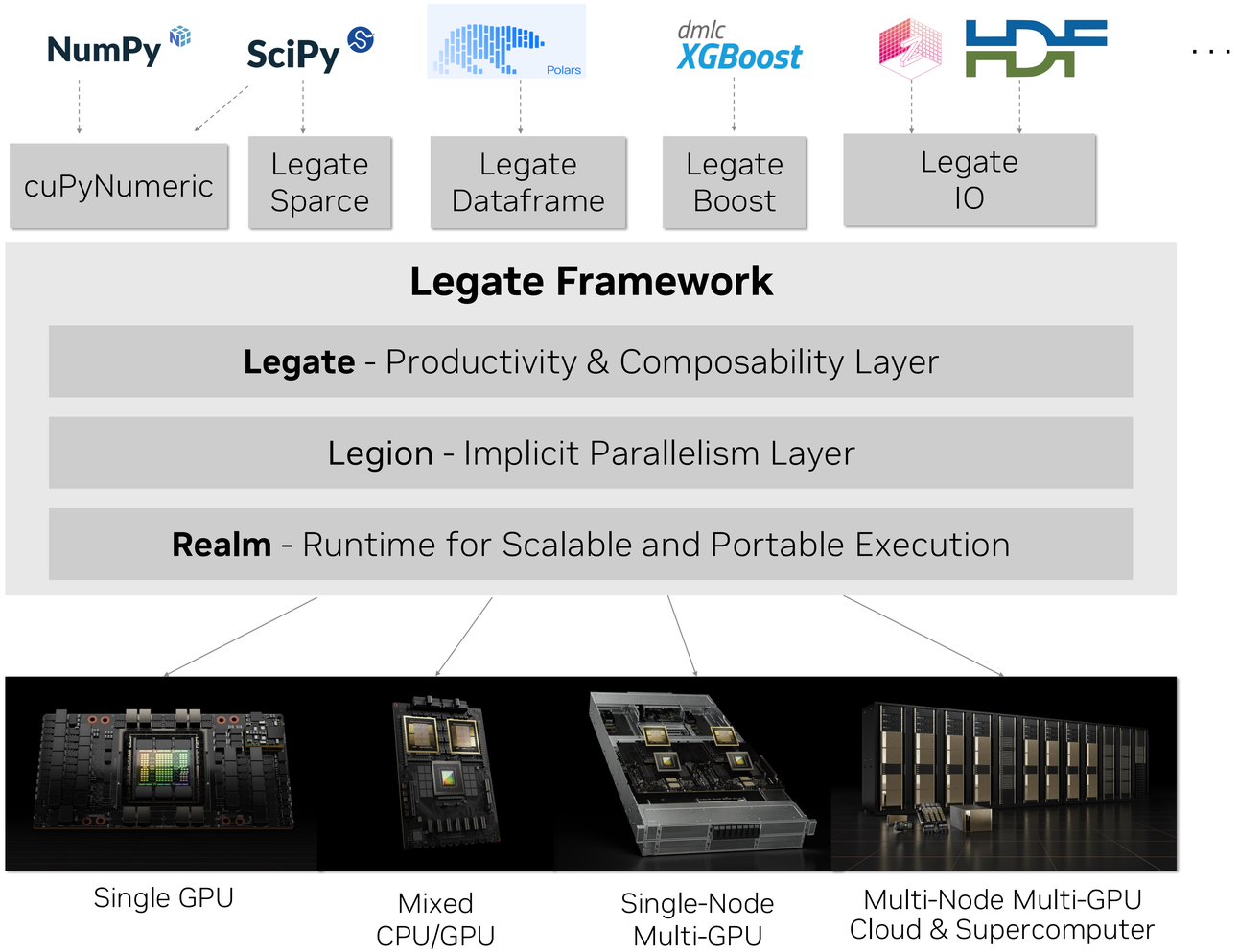
Transparently scalable libraries built on a common foundation

Libraries for “zero code change” scaling

- “Author programs in familiar APIs, scale to any machines”

Common framework for scalable libraries

- Data partitioning and coherence management
- Compute partitioning and scalable execution
- Composability / interoperability



Dr. Irina Demeshko
Malte Foerster

Dr. Quynh L Nguyen
Seshu Yamalaja

Please give it a try!

cuPyNumeric Tutorials

(https://nvda.ws/4ih3kmj)

Your feedback to

legate@nvidia.com

is highly appreciated.

Thank You!